Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 24, 2026
© 2026 Arpit Raj. All rights reserved.

Implementing Inheritance & Composition in C++

Inheritance Syntax & Constructors

Implementing inheritance in C++

```
class DerivedClass : accessSpecifier BaseClass {
    // class members
};
```

Eg

```
class Vehicle {
public:
    string name;
    string brand;
    int no_of_tyres;

    void start() { }
    void stop() { }
};

class Car : public Vehicle {
public:
    int no_of_doors;
    void startac() { }
};
```

now within car class we need constructor too, so we write all parameters be it cars specific attributes or be it general vehicle attributes, and we need to initialize vehicle attribute parameters.

```
Car(string name, string brand, int not, int nod) : Vehicle(name, brand, not) {
    this->no_of_doors = nod;
} // -> goes inside Car class
```

`Vehicle(name, brand, not)` is used to initialize parent class attributes which are general to car.

Similarly inside Vehicle class we need to write our general constructor.

```
Vehicle(string name, string brand, int not) {
    this->name = name
    this->brand = brand
    this->no_of_tyres = not
}
```

now inside main I can write

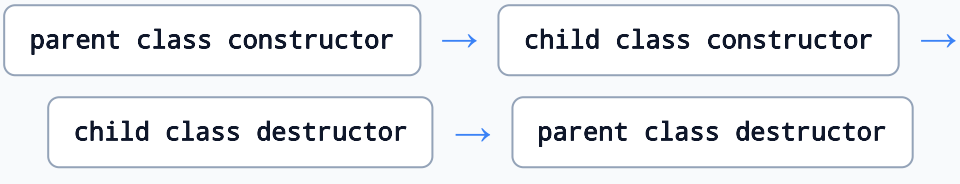
```
Car A("BMW X7", "BMW", 4, 4);
```

Note: as soon as I call this, first of all vehicle constructor is called first only after that car class constructor is called.

> 💡 **Good Practice** It is a good practice to use `protected` for parent classes members and `public` for inheritance type.

Order of Execution

Order of call



Note: let Dev be a derived class of employee

```
private: string plong
public:
    Dev(string empname, int id, string lang) : Employee(empname, id) {
        this->plang = lang
    }
```

can also be written as

```
Dev(string empname, int id, string lang) : Employee(empname, id), plang(lang) {
}
```

Mode of inheritance decides that once a member is inherited, how can it be accessed.

Multiple & Multi-Level Inheritance

Multiple inheritance Example
let there be a child who inherits properties from both father and mother and also has his unique properties.

```
class Child : public Arpit, public Aadya {
public:
    Child(string surname, int hairrgb, int eyergb, int age, bool smokes)
        : Arpit(surname), Aadya(hairrgb, eyergb) {
        this->age = age;
        this->smokes = smokes;
    }
};
```

flow of multi level inheritance
let `P -> C1 -> C2`
now from main we need `C2` info

> - C2 calls C1 calls P → ctor of P → ctor of C1 → ctor of C2
> - dtor of C2 → dtor of C1 → dtor of P

Formal Definition & Subobjects

Formal definition of inheritance: an OOP mechanism in which a derived class acquires accessible members and methods from a base class and can extend/specialize them.

NOTE #1: PRIVATE BASE CLASS MEMBERS

Never say, private base class members are not part of derived object, they are part but direct access is restricted.

NOTE #2: IS A RELATIONSHIP

IS A relationship grants that an object of derived class can also be treated as an object of base class.

Practical eg

```
class Dog : public Animal {
public:
    void bark() { }
};

void feedanimal(Animal &a) {
    a.eat();
}
```

the function expects `Animal` object because `eat` function is inside parent class (`Animal` class) but even if write `feedanimal(d)` where `d` is `Dog d`, it's allowed, because `Dog` is child of animal.

Note: a derived object doesnt copy the base class source code into derived class, a derived object contains a base class subobject.

We can assign a derived object to a parent object:

BAAP ki jagah beta allowed he, beta ki jagah baap allowed nahi

DOG is A animal, animal is not necessarily dog

© Arpit Raj, LNMIIT Jaipur

Composition vs Inheritance

- **If there is "IS A" relationship prefer inheritance**
- **If there is "HAS A" relation prefer composition aka aggregation**

Eg let there be a car and an engine (Has A relation)

```
class Engine {
public:
    // methods and properties, let
    void start() { }
};

class Car {
private:
    Engine eng;
public:
    void startcar() {
        eng.start();
    }
};
```